Vergelijking van REST en GraphQL voor IoT sensordata: verschillen in netwerkgebruik en serverbelasting

Abdullah Al Kathiry

March 2026

Contents

1 Samenvatting **4**

2 Introductie **4**

3 Literatuuronderzoek **5**
3.1 REST 5
3.2 GraphQL 5
3.3 API's binnen IoT systemen 5
3.4 Vergelijking 6
3.5 Kritische evaluatie van de bronnen 6

4 Onderzoeksvraag **6**
4.1 Research gap 6
4.2 Onderzoeksvraag 7

5 Onderzoeksopzet en methodes **7**
5.1 Deelvragen 7
5.2 Methoden 8
5.2.1 Deelvraag 1 8
5.2.2 Deelvraag 2 8
5.3 Benodigde middelen 8
5.3.1 Prototyping 8
5.3.2 Kwantitatieve data analyse: 9
5.4 Betrouwbaarheid en validiteit van de middelen 9
5.5 Prototype 9
5.5.1 Waarom .NET? 10
5.6 GitHub 11

6 Resultaten **12**
6.1 Deelvraag 1: 12
6.2 Deelvraag 2: 13

7 Discussie **14**
7.1 Deelvraag 1 14
7.1.1 Interpretatie van resultaten 14
7.1.2 Beperkingen en validiteit van het onderzoek 14
7.1.3 Betrouwbaarheid en bruikbaarheid 15
7.2 Deelvraag 2 16
7.2.1 Interpretatie van resultaten 16
7.2.2 Beperkingen en validiteit van het onderzoek 16
7.2.3 Betrouwbaarheid en bruikbaarheid 17

8 Conclusies en aanbevelingen **18**
8.1 Deelvraag 1 - Conclusie 18
8.2 Deelvraag 2 - Conclusie 18
8.3 Einde conclusie 18
8.4 Aanbevelingen 19
8.4.1 Methodologisch 19
8.4.2 Praktisch 19

9 Bronnen **20**

1 Samenvatting

Moderne IoT systemen sturen continu sensordata naar een backend. Voor deze communicatie worden API architecturen zoals REST en GraphQL gebruikt. Uit de resultaten blijkt dat de keuze van een API invloed kan hebben op prestaties zoals netwerkgebruik en serverbelasting, vooral bij grote hoeveelheden data en veel gelijktijdige sensoren.

Het doel van dit onderzoek is om te vergelijken hoe REST en GraphQL presteren in een IoT backend. Hiervoor zijn twee prototypes gebouwd die dezelfde sensordata verwerken, elk met een andere API architectuur. Met behulp van tools zoals Grafana k6 en Docker zijn metingen uitgevoerd op het gebied van netwerkgebruik en serverbelasting.

De resultaten laten zien dat REST over het algemeen efficiënter is dan GraphQL binnen deze IoT use case. GraphQL verzendt aanzienlijk meer data per seconde en gebruikt meer geheugen op de server, terwijl het CPU gebruik slechts iets hoger ligt dan bij REST. Deze verschillen blijven zichtbaar bij toenemende belasting.

Op basis van deze resultaten kan worden geconcludeerd dat REST de meest efficiënte keuze is voor het verwerken en verzenden van IoT sensordata wanneer netwerkbandbreedte en serverresources een belangrijke rol spelen. De uitkomsten van dit onderzoek helpen ontwikkelaars bij het maken van een onderbouwde keuze voor een geschikte API architectuur.

2 Introductie

Internet of Things systemen worden steeds vaker toegepast in domeinen zoals industrie en gezondheidszorg. In deze systemen sturen sensoren continu data naar een backend, waar deze wordt verwerkt en opgeslagen. Voor de communicatie tussen sensoren en backend worden verschillende API architecturen gebruikt.

Bij IoT systemen met een groot aantal sensoren spelen prestaties zoals netwerkgebruik en serverbelasting een belangrijke rol. Vooral bij continue datastromen heeft de keuze van een API architectuur invloed op de efficiëntie en schaalbaarheid van het systeem. Daarnaast kunnen hogere resourcekosten direct leiden tot hogere operationele kosten. Daarom is deze keuze belangrijk voor de prestaties en de kosten van het systeem.

Hoewel er onderzoek bestaat naar API architecturen, is er nog weinig praktisch onderzoek dat deze architecturen met elkaar vergelijkt binnen een IoT context met continue sensordata.

Dit onderzoek richt zich daarom op het vergelijken van REST en GraphQL binnen een IoT system, om inzicht te geven in de verschillen in netwerkgebruik en serverbelasting. Op basis van deze resultaten kunnen ontwikkelaars beter bepalen welke keuze in de praktijk het meest geschikt is.

3 Literatuuronderzoek

Dit is een literatuur ondrzoek over het vergelijken van REST en GraphQL, op basis van de geselecteerde wetenschappelijke artikelen (zie bronnenlijst). De artikelen kijken vooral naar verschillen in netwerkgebruik en serverbelasting. Deze factoren zijn belangrijk voor systemen waar veel data wordt verstuurd naar web services.

3.1 REST

REST is een bekend API architecture voor het bouwen van web API's en gebruikt HTTP en JSON om data te versturen. Volgens onderzoek is REST eenvoudig te implementeren en wordt het breed ondersteund door veel programmeertalen (Duduś & Maślanka, 2023). Een nadeel van REST is dat het relatief veel netwerkverkeer kan veroorzaken, omdat data in tekstformaat wordt verstuurd. Hierdoor kunnen berichten groter zijn en kan de latency hoger worden wanneer er veel requests zijn (Suryotrisongko et al., 2020).

3.2 GraphQL

GraphQL maakt het mogelijk dat een client precies de benodigde data opvraagt in een request. Door GraphQL kan het aantal requests worden verminderd en kan het netwerkgebruik lager zijn (Duduś & Maślanka, 2023). Een nadeel van GraphQL is dat complexe queries meer serverbelasting kunnen veroorzaken, omdat de server eerst de query moet verwerken voordat de data wordt teruggestuurd.

3.3 API's binnen IoT systemen

Binnen IoT systemen spelen API's een belangrijke rol bij de communicatie tussen sensoren en backend systemen. RESTful API's worden hierbij veel gebruikt, omdat ze gebruikmaken van standaard webtechnologieën zoals HTTP en eenvoudig te implementeren zijn (Maurya, 2021).

Het onderzoek laat ook zien dat API's in IoT systemen belangrijk zijn voor het koppelen van apparaten aan cloudservices en voor het toegankelijk maken van data voor gebruikers en applicaties (Maurya, 2021).

3.4 Vergelijking

Uit de literatuur blijkt dat REST is eenvoudiger en wordt daarom nog veel gebruikt. GraphQL kan het aantal requests verminderen, maar kan bij complexe queries meer serverbelasting veroorzaken.

3.5 Kritische evaluatie van de bronnen

Studies zoals (Duduś & Maślanka, 2023) en (Suryotrisongko et al., 2020) vergelijken API prestaties en zijn relevant voor dit onderzoek. Deze richten zich echter vooral op microservices en minder op IoT, wat de toepasbaarheid beperkt.

Ook (Mikhaylov et al., 2020) en (Bhole, 2025) bieden inzichten in communicatieprotocollen, maar hun focus ligt niet direct op IoT of op de vergelijking tussen REST en GraphQL.

De studie van (Maurya, 2021) is het meest relevant, omdat deze specifiek ingaat op het gebruik van API's binnen IoT systemen. Deze bron is echter vooral beschrijvend en bevat weinig prestatievergelijkingen.

4 Onderzoeksvraag

4.1 Research gap

In bestaande literatuur worden API architecturen zoals REST en GraphQL vaak vergeleken binnen traditionele webservices of microservices. Deze onderzoeken kijken vooral naar prestaties zoals netwerkgebruik en serverbelasting (Mikhaylov et al., 2020; Suryotrisongko et al., 2020).

Deze studies geven goede inzichten, maar worden meestal uitgevoerd in omgevingen met minder en onregelmatige requests. Hierdoor passen de resultaten niet altijd goed bij IoT systemen.

In IoT systemen sturen veel apparaten namelijk continu kleine hoeveelheden sensordata naar een backend. Dit zorgt voor een constante stroom van requests. Dit is anders dan bij webservices, waar vaak minder regelmatige requests worden verstuurd, zoals bij CRUD operaties.

Er is echter weinig recent onderzoek dat REST en GraphQL op een praktische manier vergelijkt binnen een IoT context met deze continue datastromen.

Daarom richt dit onderzoek zich op het vergelijken van deze API architecturen binnen een gesimuleerde IoT omgeving. Hierbij wordt gekeken naar prestaties zoals netwerkgebruik en serverbelasting bij het verwerken van sensordata.

4.2 Onderzoeksvraag

Om deze research gap te adresseren, richt dit onderzoek zich op de volgende onderzoeksvraag: In hoeverre verschillen REST en GraphQL als backend API architecturen in bandbreedtegebruik tussen IoT apparaten en de backend, en in server performance bij het verwerken van IoT sensordata?

5 Onderzoeksopzet en methodes

5.1 Deelvragen

Om de onderzoeksvraag te beantwoorden, gaan we de volgende deelvragen onderzoeken:

- In hoeverre verschillen REST en GraphQL in bandbreedtegebruik tussen IoT apparaten en een backend API bij het verzenden van IoT sensorgegevens?
- In hoeverre verschillen REST en GraphQL in serverbelasting (CPU gebruik en geheugengebruik) wanneer ze worden gebruikt als backend API voor het verwerken van IoT sensor data van IoT apparaten?

Deze deelvragen richten zich op twee specifieke prestatieaspecten: netwerkgebruik en serverbelasting. Door deze twee aspecten te meten en te vergelijken tussen REST en GraphQL bij een backend API die IoT sensordata ontvangt en verwerkt, kan worden bepaald hoe groot de verschillen in prestaties tussen deze API architecturen zijn. De resultaten van de deelvragen samen geven uiteindelijk antwoord op de hoofd onderzoeksvraag.

5.2 Methoden

5.2.1 Deelvraag 1

- Prototyping
- Data analyse

We hebben voor deze methoden gekozen omdat:

Prototyping: omdat met een prototype kan worden getest hoeveel netwerkdata wordt gebruikt bij het versturen en verwerken van IoT sensordata met REST en GraphQL en gRPC. Door dezelfde testomgeving te gebruiken kan het netwerkgebruik van de protocollen eerlijk met elkaar worden vergeleken.

Kwantitatieve data analyse: omdat de hoeveelheid verzonden en ontvangen data uit de testen moet worden geanalyseerd. Door deze gegevens te verzamelen en te vergelijken kan worden vastgesteld welk architectuur het meest efficiënt met netwerkgebruik omgaat.

5.2.2 Deelvraag 2

- Prototyping
- Data analyse

We hebben voor deze methoden gekozen omdat: **Prototyping**: omdat met een prototype kan worden gemeten hoeveel serverresources nodig zijn bij het verwerken van IoT sensordata met REST en GraphQL. Hierbij wordt gekeken naar CPU gebruik en geheugenverbruik van de server.

Kwantitatieve data analyse: omdat de verzamelde meetgegevens moeten worden geanalyseerd om verschillen tussen de architecturen inzichtelijk te maken. Door deze gegevens te vergelijken kan worden vastgesteld welk architectuur de minste serverbelasting veroorzaakt.

5.3 Benodigde middelen

5.3.1 Prototyping

Docker: Om de REST en GraphQL services in dezelfde omgeving te laten draaien. Hierdoor kunnen de architecturen onder gelijke omstandigheden worden getest.

k6: Om IoT sensoren te simuleren en grote hoeveelheden requests naar de backend te sturen.

5.3.2 Kwantitatieve data analyse:

Python : Om prestatiegegevens van de containerized backend API te verzamelen , zoals CPU gebruik en geheugen, door middel van de Docker Containers API, en deze gegevens te exporteren naar Excel bestanden voor verdere analyse.

Excel: Om de verzamelde gegevens van de Docker API en K6 te verwerken en te analyseren in datasets, tabellen en grafieken, zodat de verschillen in prestaties tussen REST en GraphQL duidelijk zichtbaar worden.

5.4 Betrouwbaarheid en validiteit van de middelen

Met deze middelen kan een gecontroleerde testomgeving worden opgezet waarin alle architecturen onder dezelfde omstandigheden worden getest. Dit zorgt dat de metingen vergelijkbaar en reproduceerbaar zijn. Daarnaast maken de gebruikte tools het mogelijk om meetgegevens automatisch te verzamelen en overzichtelijk weer te geven. Op deze manier kan de data nauwkeurig worden geanalyseerd en kunnen betrouwbare conclusies worden getrokken.

5.5 Prototype

Voordat we beginnen met het presenteren van de resultaten en de testen die we hebben uitgevoerd om de deelvragen te beantwoorden, geven we eerst een overzicht van de architectuur van het prototype. Dit prototype is namelijk gebruikt bij alle deelvragen als bron van data. Zoals eerder uitgelegd, hebben we k6 gebruikt om IoT sensoren te simuleren door HTTP requests te genereren. Daarna hebben we zowel een REST als een GraphQL API gebouwd met hetzelfde framework, namelijk .NET. We hebben daarbij dezelfde businesslogica en dezelfde ORM gebruikt in beide API's, zodat ze zo vergelijkbaar mogelijk zijn.

Daarnaast hebben we Docker gebruikt voor containerisatie en voor observability, onder andere met docker stats. We hebben ook twee Python scripts gemaakt om data uit docker stats te importeren en deze vervolgens naar Excel te exporteren voor verdere analyse. Dit alles hebben we uitgevoerd op dezelfde infrastructuur, om de objectiviteit zoveel mogelijk te waarborgen en om reproduceerbare en betrouwbare data te verkrijgen.

5.5.1 Waarom .NET?

Voor de ontwikkeling van de API's hebben we gekozen voor het .NET 9 framework. Deze keuze hebben we gemaakt omdat .NET wordt onderhouden door engineers van Microsoft en veel wordt gebruikt in enterprise software. Dit betekent dat het actief wordt ondersteund door ervaren ontwikkelaars wereldwijd (DotNet Foundation, z.d.). Daarnaast is .NET erg snel in vergelijking met bekende frameworks zoals Java Spring en Node.js (TechEmpower, z.d.). Omdat we in ons onderzoek ook kijken naar bruikbaarheid, vinden we deze eigenschappen belangrijk en passend voor onze studie.

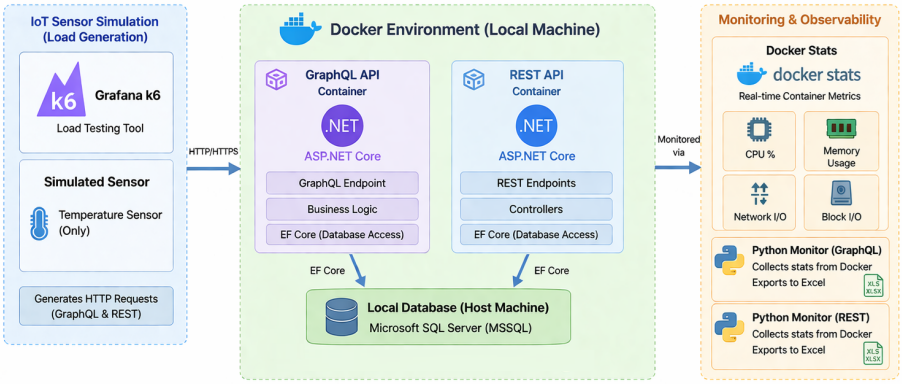

Figure 1: Algemeen overzicht van de architectuur van het prototype.

Om de structuur van beide API architecturen te verduidelijken, is hieronder een voorbeeld opgenomen van het toevoegen van een temperatuurmeting in zowel GraphQL als REST.

GraphQL – Mutatie

```
mutation {
  addTemperatureReading(input: {
    sensorName: "sensor-1",
    temperatureC: 22.5,
    readingTimeUtc: "2026-03-26T03:00:00Z"
  }) {
    id
  }
}
```

Response:

```
{
  "data": {
    "addTemperatureReading": {
      "id": 2775399
    }
  }
}
```

REST – POST endpoint

Endpoint:

```
POST /api/Temperature
```

Request body:

```
{
  "SensorName": "Sensor1",
  "TemperatureC": 23.5,
  "ReadingTimeUtc": "2026-03-25T12:00:00Z"
}
```

Response:

```
{
  "message": "Temperature reading saved successfully.",
  "id": 2775398
}
```

5.6 GitHub

Voor meer informatie over de prototype , zoals de k6 scripts en de code van GraphQL en REST API's, is alles beschikbaar in deze GitHub repository : `https://github.com/lateNightCoder200/Onderzoek.git`

6 Resultaten

6.1 Deelvraag 1:

In hoeverre verschillen REST en GraphQL in bandbreedtegebruik tussen IoT apparaten en een backend API bij het verzenden van IoT sensorgegevens?

We hebben drie testscenario's uitgevoerd waarbij IoT sensoren zijn gesimuleerd met 100, 500 en 800 sensoren gedurende 10 minuten, om het bandwidth van REST en GraphQL te meten. Deze scenario's zijn gekozen omdat ze verschillende schaalniveaus representeren, van een kleine tot een meer realistische en zwaardere load. Hierdoor is het mogelijk om te analyseren hoe beide API architecturen presteren bij toenemende load en om verschillen in efficiëntie duidelijk zichtbaar te maken.

Test-1: 100 sensors

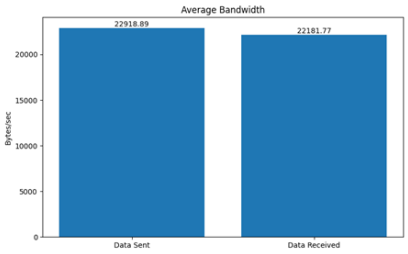

Figure 2: REST - 100 sensors

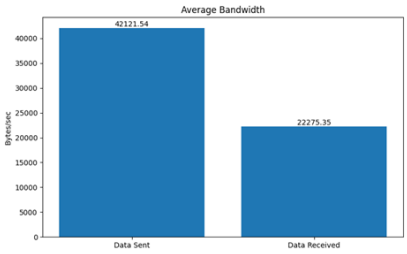

Figure 3: GraphQL - 100 sensors

Test-2: 500 sensors

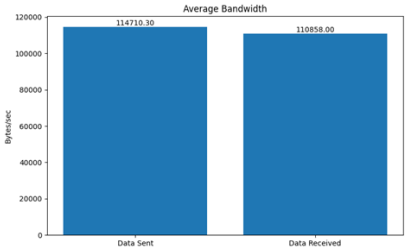

Figure 4: REST - 500 sensors

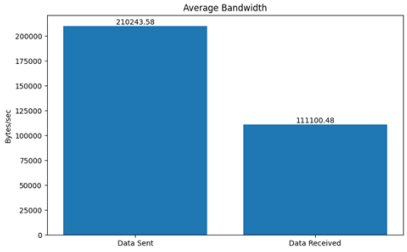

Figure 5: GraphQL - 500 sensors

Test-3: 800 sensors

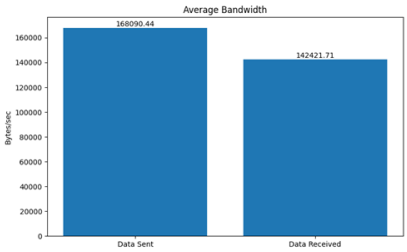

Figure 6: REST - 800 sensors

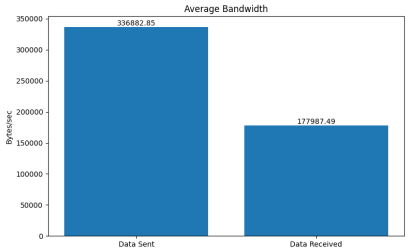

Figure 7: GraphQ - 800 sensors

6.2 Deelvraag 2:

In hoeverre verschillen REST en GraphQL in serverbelasting (CPU gebruik en geheugengebruik) wanneer ze worden gebruikt als backend API voor het verwerken van IoT sensor data van IoT apparaten?

We hebben drie testscenario's uitgevoerd waarbij IoT sensoren zijn gesimuleerd met 100, 500 en 800 sensoren gedurende 10 minuten, om de server performance van REST en GraphQL te meten. In deze scenario's zijn het CPU gebruik, het geheugengebruik in MB en het percentage geheugengebruik vastgelegd. Deze scenario's zijn gekozen omdat ze verschillende schaalniveaus representeren, van een kleine tot een meer realistische en zwaardere load. Dit maakt het mogelijk om te analyseren hoe beide API architecturen presteren bij toenemende load en om verschillen in resourcegebruik duidelijk zichtbaar te maken.

	CPU gebruik	Geheugengebruik (MB)	Geheugengebruik (%)
REST	1.44%	86.08	1.09%
GraphQL	1.40%	200.75	2.54%

Table 1: Test-1: 100 sensors: Vergelijking van CPU- en geheugengebruik tussen REST en GraphQL

	CPU gebruik	Geheugengebruik (MB)	Geheugengebruik (%)
REST	5.36%	126.65	1.60%
GraphQL	5.70%	236.16	2.99%

Table 2: Test-2: 500 sensors: Vergelijking van CPU- en geheugengebruik tussen REST en GraphQL

	CPU gebruik	Geheugengebruik (MB)	Geheugengebruik (%)
REST	8.35%	144.75	1.83%
GraphQL	9.13%	260.46	3.30%

Table 3: Test-3: 800 sensors: Vergelijking van CPU- en geheugengebruik tussen REST en GraphQL

7 Discussie

7.1 Deelvraag 1

7.1.1 Interpretatie van resultaten

Data sent is hoger bij GraphQL: De resultaten tonen dat GraphQL aanzienlijk meer data per seconde verzendt dan REST. Dit betekent dat sensoren meer netwerkverkeer genereren richting de backend. Dit komt overeen met literatuur, waarin REST wordt beschreven als efficiënter in netwerkgebruik (K. Duduś & K. Maślanka, 2023; K. Mikhaylov et al., 2020).

Data received is vergelijkbaar bij lage load: Bij lage load is de ontvangen data vergelijkbaar voor beide technologieën. Dit betekent dat er weinig verschil is in response grootte bij eenvoudige IoT data, wat overeenkomt met bestaande onderzoeken (H. Suryotrisongko et al., 2020).

Verschil groeit bij hoge load: Bij hogere load ontvangt GraphQL meer data dan REST. Dit laat zien dat verschillen toenemen bij schaal, wat ook in literatuur wordt bevestigd (H. Suryotrisongko et al., 2020).

Oorzaken

Wat het hogere bandbreedtegebruik bij GraphQL in de testresultaten verklaart, is dat de gebruikte GraphQL mutaties maken dat sensoren meer data per request naar de backend sturen dan bij REST in deze testen. Dit komt doordat een GraphQL request naast de sensordata ook extra structuur bevat, zoals de mutation en extra velden in de response. Daardoor is de grootte van zowel de request als de response groter dan bij REST, waar meestal alleen een JSON body wordt verstuurd.

7.1.2 Beperkingen en validiteit van het onderzoek

De uitgevoerde tests zijn gebaseerd op gesimuleerde sensoren met behulp van k6, waardoor de resultaten mogelijk afwijken van een realistische IoT omgev-

ing. In een praktijkomgeving spelen verschillende netwerkfactoren een rol die in deze simulatie niet zijn meegenomen, zoals pakketverlies, netwerkvertraging, variabele bandbreedte en de invloed van tussenliggende systemen, zoals message queues of gateways.

Deze factoren kunnen in de praktijk een significante invloed hebben op de hoeveelheid verzonden en ontvangen data, en daarmee op de uiteindelijke prestaties van de API architectuur.

Verder is de schaal van de experimenten beperkt gebleven tot maximaal 800 virtuele sensoren. Hoewel dit voldoende is om trends te identificeren, weerspiegelt dit niet de schaal van grootschalige IoT systemen, waarin vaak duizenden apparaten actief zijn.

7.1.3 Betrouwbaarheid en bruikbaarheid

De betrouwbaarheid van dit onderzoek wordt ondersteund door het gebruik van consistente testomstandigheden. Beide API's zijn gebouwd met hetzelfde framework en beide API prototypes zijn getest onder identieke scenario's met k6 en met dezelfde hoeveelheid sensordata. Dit zorgt voor een eerlijke vergelijking tussen REST en GraphQL.

Daarnaast zijn de metingen uitgevoerd over een vaste tijdsduur en met een oplopend aantal virtuele sensoren (100, 500 en 800), wat bijdraagt aan de reproduceerbaarheid van de resultaten.

Verder zijn alle metingen meerdere keren uitgevoerd en op dezelfde infrastructuur getest, zodat externe factoren zoveel mogelijk zijn beperkt. Dit verkleint de kans dat de resultaten worden beïnvloed door toevallige variaties in de testomgeving.

De bruikbaarheid van deze testen ligt voornamelijk in het verkrijgen van inzicht in de bandwidth van REST en GraphQL binnen een IoT context. De resultaten geven een indicatie dat REST efficiënter omgaat met netwerkverkeer bij het verzenden van data, met name bij hogere load.

Deze inzichten kunnen worden toegepast bij het ontwerpen van IoT systemen waarin bandwidth een kritische factor is, zoals bij grootschalige sensoromgevingen of netwerken met beperkte capaciteit.

7.2 Deelvraag 2

7.2.1 Interpretatie van resultaten

CPU gebruik is vergelijkbaar, maar GraphQL iets hoger : De resultaten tonen dat het CPU gebruik van REST en GraphQL dicht bij elkaar ligt in alle tests, maar dat GraphQL consequent iets hoger scoort. Dit betekent dat beide technologieën vergelijkbare verwerkingscapaciteit vereisen, met een lichte extra belasting voor GraphQL. Dit komt overeen met literatuur waarin wordt aangegeven dat verschillen in CPU gebruik tussen API technologieën vaak beperkt zijn (K. Duduś & K. Maślanka, 2023).

Geheugengebruik is significant hoger bij GraphQL : Het geheugengebruik ligt bij GraphQL in alle tests duidelijk hoger dan bij REST, ongeveer een verdubbeling. Dit betekent dat de GraphQL API meer geheugen nodig heeft om dezelfde hoeveelheid IoT data te verwerken. Dit sluit aan bij bestaande onderzoeken waarin wordt beschreven dat flexibelere API structuren vaak meer resources vereisen (K. Mikhaylov et al., 2020).

Verschillen blijven consistent bij hogere load : Naarmate het aantal virtual users toeneemt van 100 naar 800, nemen CPU en geheugengebruik bij beide technologieën toe, maar blijft het relatieve verschil tussen REST en GraphQL grotendeels gelijk. Dit betekent dat GraphQL structureel meer resources gebruikt, ook bij hogere schaal.

Oorzaken

Resultaten kan worden verklaard doordat GraphQL extra stappen uitvoert bij het verwerken van requests. De server moet eerst de query analyseren, valideren en vervolgens per veld bepalen welke data moet worden opgehaald. De response wordt ook vaak opgebouwd uit geneste structuren. Deze extra verwerking zorgt voor meer gebruik van geheugen en ook voor een hoger CPU gebruik.

7.2.2 Beperkingen en validiteit van het onderzoek

De meeste beperkingen van dit onderzoek komen overeen met de eerder genoemde punten in de discussie onder deelvraag 2. Zo zijn de tests gebaseerd op simulaties met behulp van k6, waardoor belangrijke factoren uit de praktijk ontbreken. Denk hierbij aan netwerkvertraging, pakketverlies en de invloed van tussenliggende systemen, zoals message queues of gateways.

Deze factoren bepalen in de praktijk hoeveel data door de API wordt verwerkt en daarmee ook de benodigde serverbelasting voor het analyseren van deze data. Bijvoorbeeld kan een infrastructuur die sensoren ondersteunt bij het be-

trouwbaar verzenden van grote hoeveelheden data leiden tot een hogere serverbelasting (CPU en geheugengebruik) in vergelijking met een minder robuuste infrastructuur. Daarom wordt aangenomen dat het ontbreken van deze factoren invloed heeft op de nauwkeurigheid van de resultaten.

Er zijn factoren die kunnen beïnvloeden. De API's zijn namelijk uitgevoerd binnen Docker containers op een lokale machine, wat kan afwijken van productieomgevingen waarin vaak gebruik wordt gemaakt van cloudinfrastructuur. Verschillen in hardware, resource allocatie en containerconfiguraties kunnen invloed hebben op het gemeten CPU en geheugengebruik. Hierdoor kunnen de verkregen resultaten een beperkte representatie vormen van prestaties in een realistische productieomgeving.

7.2.3 Betrouwbaarheid en bruikbaarheid

De betrouwbaarheid van dit onderzoek wordt ondersteund door het gebruik van consistente testomstandigheden. Beide API prototypes zijn ontwikkeld binnen dezelfde .NET framework en gedeployed in Docker containers. Daarnaast maken beide API's gebruik van dezelfde businesslogica, ORM en database, waardoor verschillen in de resultaten niet worden veroorzaakt door variaties in de implementatie, maar door de API architectuur zelf. Hierdoor draaien beide API's onder vergelijkbare omstandigheden, wat zorgt voor een eerlijke vergelijking van CPU en geheugengebruik.

Daarnaast zijn de metingen uitgevoerd over een vaste tijdsduur en bij een oplopend aantal virtuele sensors, wat bijdraagt aan de reproduceerbaarheid van de resultaten. Door deze gecontroleerde opzet wordt de invloed van externe factoren zoveel mogelijk beperkt.

De bruikbaarheid van dit onderzoek ligt in het verkrijgen van inzicht in de serverbelasting van REST en GraphQL, met name op het gebied van CPU en geheugengebruik. Deze inzichten helpen bij het begrijpen hoe beide API architecturen omgaan met toenemende loads en welke impact dit heeft op serverresources. Dit is vooral relevant voor IoT systemen, waarin vaak grote hoeveelheden data continu worden verwerkt en efficiënt resourcegebruik belangrijk is. Efficiënt gebruik van resources draagt daarnaast bij aan lagere operationele kosten en een duurzamer gebruik van energie, doordat minder rekenkracht en geheugen nodig zijn.

Op basis van deze resultaten kunnen ontwikkelaars beter inschatten welke API architectuur het meest geschikt is voor hun specifieke situatie. Vooral in omgevingen met beperkte servercapaciteit of hoge schaalvereisten kan deze kennis bijdragen aan het maken van een onderbouwde keuze. Dit is bijvoorbeeld relevant bij het gebruik van cloudservices, waar serverresources vaak per gebruik worden afgerekend en efficiënt resourcegebruik direct kan leiden tot lagere kosten.

8 Conclusies en aanbevelingen

8.1 Deelvraag 1 - Conclusie

Op basis van de resultaten van deelvraag 1 kan worden geconcludeerd dat GraphQL aanzienlijk meer data per seconde verzendt dan REST, met een verschil van ongeveer 80% tot 100% in alle tests. De hoeveelheid ontvangen data per seconde blijft bij lage load vrijwel gelijk, maar ligt bij hogere belasting hoger voor GraphQL.

Deze verschillen blijven consistent naarmate het aantal IoT sensoren toeneemt, wat aangeeft dat GraphQL in deze testopzet meer netwerkbandbreedte vereist. Hiermee laat het onderzoek zien dat REST efficiënter is in termen van netwerkgebruik voor het verzenden van IoT sensordata binnen deze use case.

8.2 Deelvraag 2 - Conclusie

Op basis van de resultaten van deelvraag 2 kan worden geconcludeerd dat REST en GraphQL vergelijkbaar CPU gebruik hebben, waarbij GraphQL in alle tests licht hoger uitkomt, maar dat het geheugengebruik bij GraphQL aanzienlijk hoger ligt dan bij REST, ongeveer tot twee keer zoveel in alle scenario's.

Deze verschillen blijven consistent bij toenemende loads, wat aangeeft dat GraphQL structureel meer serverresources vereist voor het verwerken van IoT sensordata. Hiermee laat het onderzoek zien dat REST efficiënter is in termen van server performance, vooral op het gebied van geheugen, terwijl de verschillen in CPU gebruik beperkt blijven.

8.3 Einde conclusie

Op basis van de conclusies kan worden geconcludeerd dat REST over het algemeen efficiënter is dan GraphQL in zowel netwerkgebruik als serverbelasting binnen deze IoT usecase. GraphQL verzendt significant meer data per seconde (ongeveer 80% tot 100% meer) en gebruikt daarnaast aanzienlijk meer geheugen op de server, terwijl het CPU gebruik slechts licht hoger ligt dan bij REST. Deze verschillen blijven consistent bij toenemende belasting, wat aangeeft dat GraphQL structureel meer resources vereist. Daarmee laat het onderzoek zien dat REST de meest efficiënte keuze is voor het verwerken en verzenden van IoT sensordata wanneer netwerkbandbreedte en serverresources een belangrijke rol spelen.

8.4 Aanbevelingen

8.4.1 Methodologisch

In toekomstig onderzoek wordt aanbevolen om tests uit te voeren in een realistischere omgeving, waarbij gebruik wordt gemaakt van bestaande IoT infrastructuren in plaats van uitsluitend gesimuleerde scenario's. Hierbij dienen factoren zoals netwerkvertraging, pakketverlies en de invloed van tussenliggende systemen (zoals message queues en gateways) te worden meegenomen, aangezien deze direct invloed hebben op zowel bandwidth als server performance.

Daarnaast is het aan te raden om de API's te deployen in cloudomgevingen in plaats van lokale Docker containers, zodat verschillen in hardware, schaalbaarheid en resource allocatie beter worden gereflecteerd. Dit is praktisch mogelijk, omdat veel cloudplatformen makkelijk toegang geven tot schaalbare testomgevingen tegen relatief lage kosten. Bijvoorbeeld kun je met Amazon Web Services EC2 of met Microsoft Azure Virtual Machines snel extra servers opzetten om tests uit te voeren en deze daarna weer uitschakelen om kosten te besparen.

8.4.2 Praktisch

Voor organisaties die IoT systemen ontwikkelen waarin bandwidth en resourcegebruik een kritische rol spelen, zoals bij grootschalige sensoromgevingen of netwerken met beperkte capaciteit, wordt aanbevolen om REST te gebruiken als API architectuur. Deze keuze is direct toepasbaar en vereist geen extra complexe infrastructuur, wat de implementatiekosten beperkt houdt.

GraphQL kan een geschikte keuze zijn in situaties waarin flexibiliteit in dataverzoeken belangrijk zijn, bijvoorbeeld bij het ophalen van sensorgegevens uit backend systemen. GraphQL queries maken het mogelijk om precies de benodigde data op te vragen. Voor het opslaan van sensordata zijn echter vaak meer stappen nodig, wat kan leiden tot hogere kosten en een lager efficiënt gebruik van resources op de lange termijn.

Deze aanbevelingen zijn niet alleen toepasbaar binnen IoT systemen, maar ook breder inzetbaar in andere softwarearchitecturen, waar een afweging gemaakt moet worden tussen performance en flexibiliteit

9 Bronnen

Bhole, M. (2025). Network-aware gRPC streaming for edge-to-cloud time-series data ingestion. FRUCT Conference Proceedings.

Duduś, K., & Maślanka, K. (2023). Performance comparison of programming interfaces for microservices architecture. IBIMA.

Mikhaylov, K., et al. (2020). Comparative analysis of the REST and gRPC used in the monitoring system of communication network virtualized infrastructure.

Suryotrisongko, H., et al. (2020). Performance comparison of communication protocols in microservices architecture. Electronics, 9(4).

Maurya, R. (2021). Application of RESTful APIs in IoT: A review. International Journal for Research in Applied Science and Engineering Technology, 9(2), 145–151.

DotNet Foundation. (z.d.). Geraadpleegd van `https://dotnetfoundation.org/`

TechEmpower. (z.d.). Geraadpleegd van `https://www.techempower.com/benchmarks/#section=data-r23\&test=json`